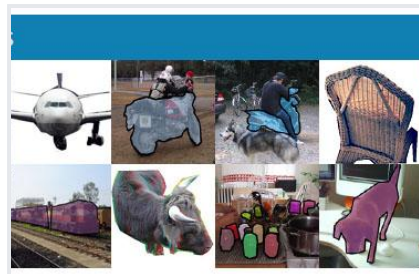
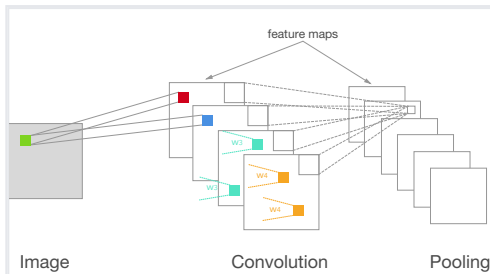


CNN: Convolutional Neural Network

이미지 구조를 이용하는 딥러닝

CNN은 픽셀을 단순한 긴 vector로만 보지 않고, 가까운 픽셀 사이의 공간적 패턴을 보존해 edge, texture, part, object를 단계적으로 학습합니다.



학습 흐름

CNN을 읽는 순서

1. 이미지를 tensor로 표현한다.
2. convolution이 local pattern을 찾는 방식을 이해한다.
3. padding, stride, channel, filter 개념을 연결한다.
4. pooling과 깊은 layer가 representation을 어떻게 바꾸는지 본다.
5. LeNet, AlexNet, VGG, ResNet의 발전 방향을 읽는다.

단원	핵심 질문
입력	이미지는 왜 3차원 tensor인가?
합성곱	filter는 무엇을 학습하는가?
구조	왜 FC보다 parameter가 적은가?
학습	어떤 augmentation과 transfer가 필요한가?

CNN은 DNN의 원리를 이미지의 공간 구조에 맞게 제한한 모델입니다.

왜 이미지에는 CNN인가

완전연결망의 문제

- 224×224 RGB 이미지를 vector로 펼치면 150,528차원입니다.
- 첫 hidden layer가 1,000개 뉴런만 가져도 weight가 약 1.5억 개 필요합니다.
- 이미지에서 가까운 픽셀은 서로 관련이 큰데, MLP는 이 공간 구조를 기본적으로 사용하지 않습니다.
- CNN은 local connectivity와 parameter sharing으로 이 문제를 줄입니다.

$$P_{\text{FC}} = (HWC_{in})C_{out} + C_{out}$$

$$P_{\text{conv}} = K_h K_w C_{in} C_{out} + C_{out}$$

convolution parameter 수는 입력 이미지의 가로·세로 크기에 직접 비례하지 않습니다.

Image Tensor

이미지는 위치와 채널을 가진다

- grayscale 이미지는 보통 $H \times W \times 1$ tensor입니다.
- RGB 이미지는 $H \times W \times 3$ tensor입니다.
- CNN은 높이, 너비, channel 구조를 유지한 채 feature map을 만듭니다.
- batch까지 포함하면 실제 입력 shape은 $B \times C \times H \times W$ 또는 $B \times H \times W \times C$ 입니다.

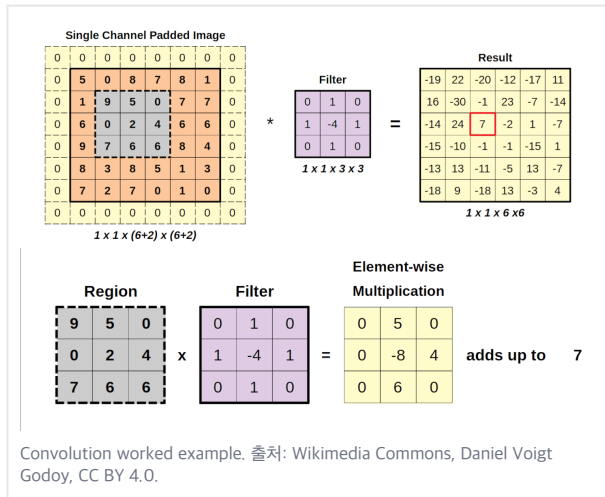
축	의미	예
H	세로 위치	224
W	가로 위치	224
C	채널	RGB 3
B	batch size	32

프레임워크마다 channel-first와 channel-last 표기가 다를 수 있으므로 모델 입력 shape을 항상 확인해야 합니다.

Local Receptive Field

작은 영역을 반복해서 본다

- convolution filter는 이미지 전체가 아니라 작은 window를 봅니다.
- 같은 filter가 이미지의 여러 위치를 훑으며 같은 패턴을 찾습니다.
- 앞쪽 layer의 receptive field는 작지만, layer가 깊어질수록 원본 이미지에서 보는 범위가 넓어집니다.



CNN은 “이미지의 어느 위치에 있는 같은 패턴은 같은 방식으로 인식한다”는 가정을 사용합니다.

Convolution Operation

filter와 입력 patch의 내적

- filter는 작은 weight tensor입니다.
- filter가 한 위치의 입력 patch와 곱해져 하나의 출력 값을 만듭니다.
- 여러 filter를 쓰면 여러 channel의 feature map이 생깁니다.
- 학습은 filter weight를 조정해 유용한 pattern detector를 만드는 과정입니다.

$$Y_{i,j,k} = b_k + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c=1}^{C_{in}} X_{i+u,j+v,c} K_{u,v,c,k}$$

실제 딥러닝 라이브러리는 수학적 convolution보다 cross-correlation 형태를 쓰는 경우가 많지만, 핵심은 local weighted sum입니다.

Filter는 무엇을 배우는가

앞쪽 layer는 낮은 수준의 패턴

- edge, corner, 색 대비, 방향성 있는 선분처럼 지역적인 패턴을 감지합니다.
- 여러 filter가 서로 다른 패턴에 반응합니다.
- activation을 통과한 결과가 feature map입니다.

뒤쪽 layer는 더 추상적인 조합

깊이	학습되는 경향
shallow	edge, color contrast, texture
middle	part, motif, local shape
deep	object-level pattern

CNN이 “feature extractor”라고 불리는 이유는 앞쪽 convolution block이 예측에 유용한 표현을 자동으로 만들기 때문입니다.

Padding과 Stride

출력 크기를 결정하는 설계값

- kernel size K 는 filter가 한 번에 보는 영역입니다.
- stride S 는 filter가 이동하는 간격입니다.
- padding P 는 가장자리에 값을 덧붙여 출력 크기와 border 정보를 조절합니다.
- stride가 커지면 feature map 크기는 줄고 계산량도 줄어듭니다.

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

모델 구현에서 shape mismatch가 나면 가장 먼저 padding, stride, kernel size를 확인합니다.

Parameter Sharing

같은 filter를 모든 위치에서 쓴다

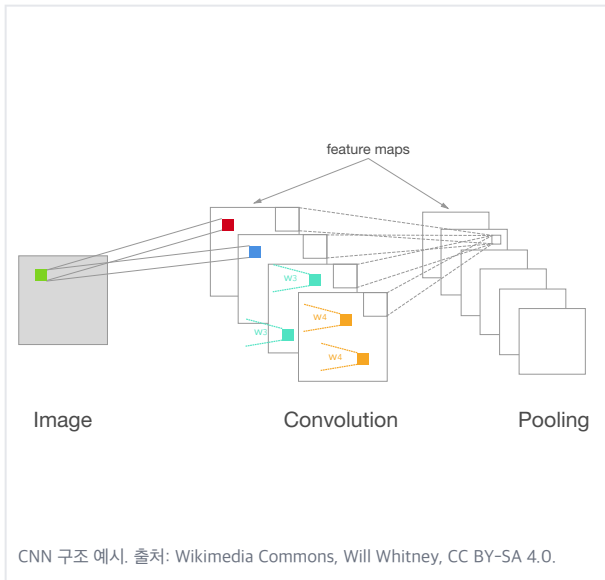
- 완전연결층은 위치마다 다른 weight를 가집니다.
- convolution은 같은 filter를 모든 위치에 공유합니다.
- 그래서 이미지 크기가 커져도 filter parameter 수는 kernel 크기, 입력 channel, 출력 channel에 의해 결정됩니다.
- 이 구조 덕분에 CNN은 이미지에서 이동한 패턴에도 비교적 잘 반응합니다.

$$P_{\text{conv}} = K_h K_w C_{in} C_{out} + C_{out}$$

예: 3×3 kernel, 입력 64채널, 출력 128채널이면
weight는 3×3×64×128개입니다.

CNN의 inductive bias는 “가까운 픽셀은 관련 있고, 같은 패턴은 위치가 달라도 의미가 있다”입니다.

Feature Map



filter 반응의 지도

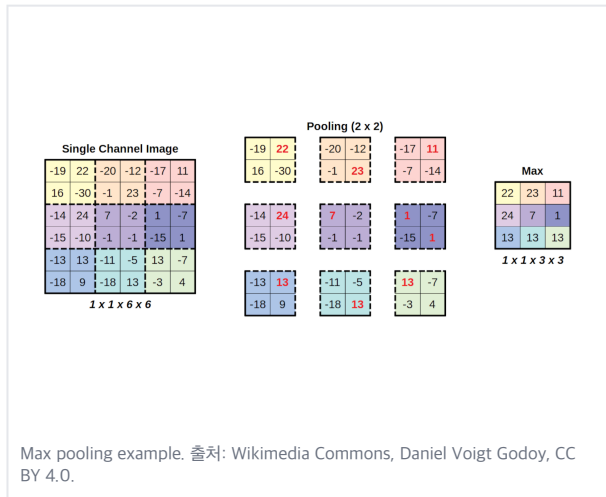
- feature map의 각 위치는 해당 위치 주변 patch가 filter와 얼마나 잘 맞는지 나타냅니다.
- channel 수는 사용한 filter 수와 같습니다.
- layer가 깊어지면 feature map의 공간 크기는 줄고 channel 수는 늘어나는 경우가 많습니다.
- 마지막에는 feature map을 classifier가 사용할 vector로 요약합니다.

Pooling

위치 변화에 조금 둔감하게 만든다

- max pooling은 작은 영역에서 가장 큰 activation만 남깁니다.
- average pooling은 평균값으로 영역을 요약합니다.
- pooling은 공간 크기와 계산량을 줄이고, 작은 이동에 대한 민감도를 낮춥니다.
- 최근 구조에서는 stride convolution이나 global average pooling으로 대체되는 경우도 많습니다.

$$Y_{i,j,c} = \max_{(u,v) \in R_{i,j}} X_{u,v,c}$$



Receptive Field

깊어질수록 원본에서 보는 범위가 넓어진다

- 첫 convolution layer의 한 activation은 원본 이미지의 작은 patch만 봅니다.
- 다음 layer는 이전 feature map의 여러 activation을 보므로 원본 기준 범위가 커집니다.
- 큰 receptive field는 전체 형태를 판단하는 데 필요합니다.
- 너무 빨리 downsampling하면 작은 객체 정보가 사라질 수 있습니다.

$$r_{\ell} = r_{\ell-1} + (k_{\ell} - 1) \prod_{m < \ell} s_m, \quad r_0 = 1$$

k 는 kernel size, s 는 이전 layer들의 stride를 뜻합니다.

CNN Block

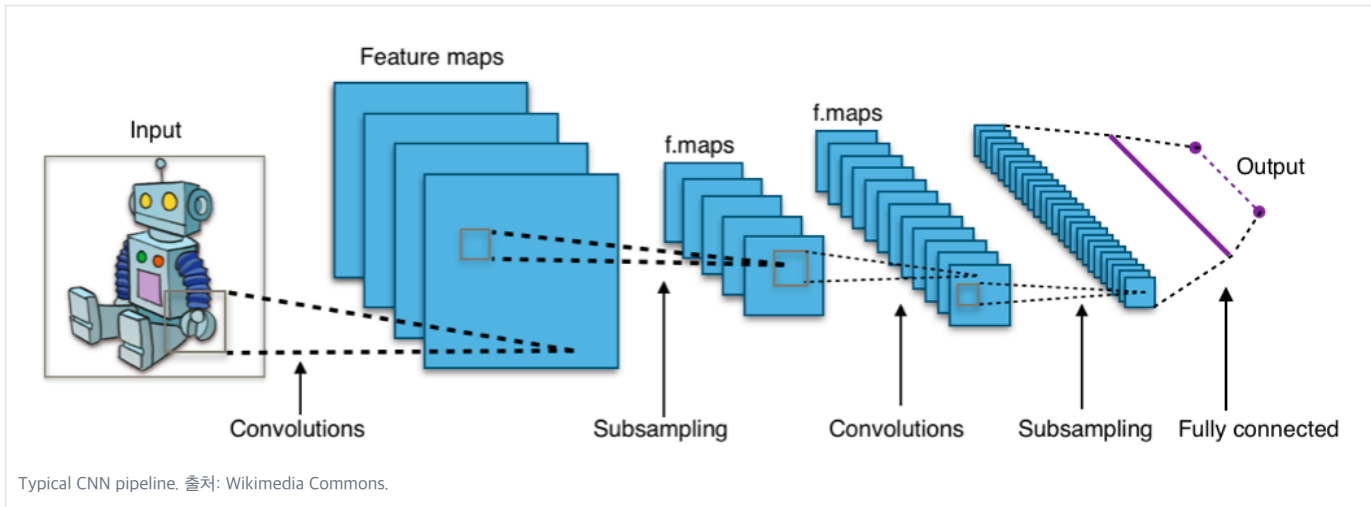
기본 블록의 반복

- Convolution: local pattern을 계산합니다.
- Normalization: activation scale을 안정화합니다.
- Activation: 비선형성을 부여합니다.
- Pooling 또는 stride: 공간 크기를 줄입니다.
- 여러 block을 쌓으면 low-level feature에서 high-level feature로 이동합니다.

단계	shape 변화 예
input	224×224×3
conv 3×3, 64	224×224×64
pooling 2×2	112×112×64
conv block	56×56×128
classifier	class probability

$$\hat{p} = \text{softmax}(W \text{ GAP}(f_{\theta}(x)) + b)$$

Classic CNN Pipeline

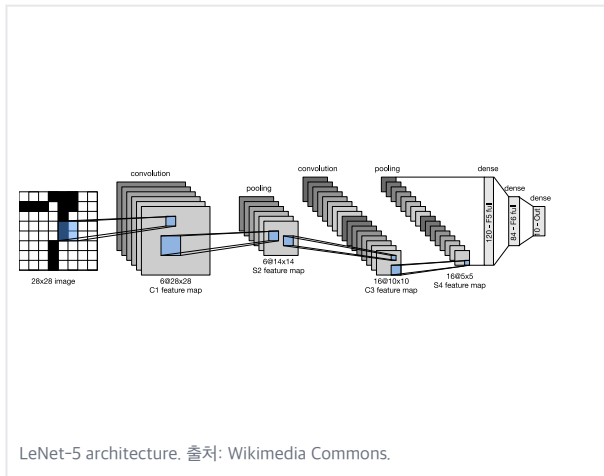


입력 이미지 → convolution/pooling으로 feature map 생성 → flatten 또는 global pooling → fully connected/classifier로 최종 클래스 예측.

LeNet-5

손글씨 숫자 인식의 고전 구조

- LeNet-5는 CNN이 실제 문자 인식 문제에서 강력할 수 있음을 보여준 대표적 구조입니다.
- convolution과 subsampling, fully connected layer를 조합했습니다.
- MNIST 같은 작은 grayscale 이미지 분류를 설명할 때 기본 출발점으로 자주 등장합니다.



MNIST와 CNN



MNIST 손글씨 숫자 예시. 출처: Wikimedia Commons, Josef Steppan, CC BY-SA 4.0.

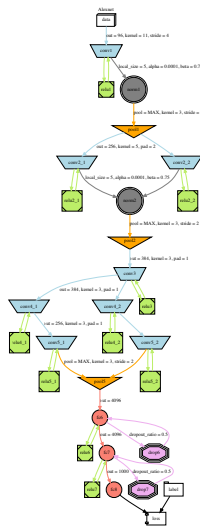
왜 CNN이 자연스러운가

- 숫자의 획은 이미지 안에서 약간 이동해도 같은 의미를 유지합니다.
- 3×3, 5×5 filter는 획의 방향과 곡선을 지역적으로 감지할 수 있습니다.
- pooling과 깊은 layer는 획 조합을 더 안정적인 숫자 표현으로 바꿉니다.
- MLP는 위치 구조를 버리지만 CNN은 구조를 보존합니다.

AlexNet

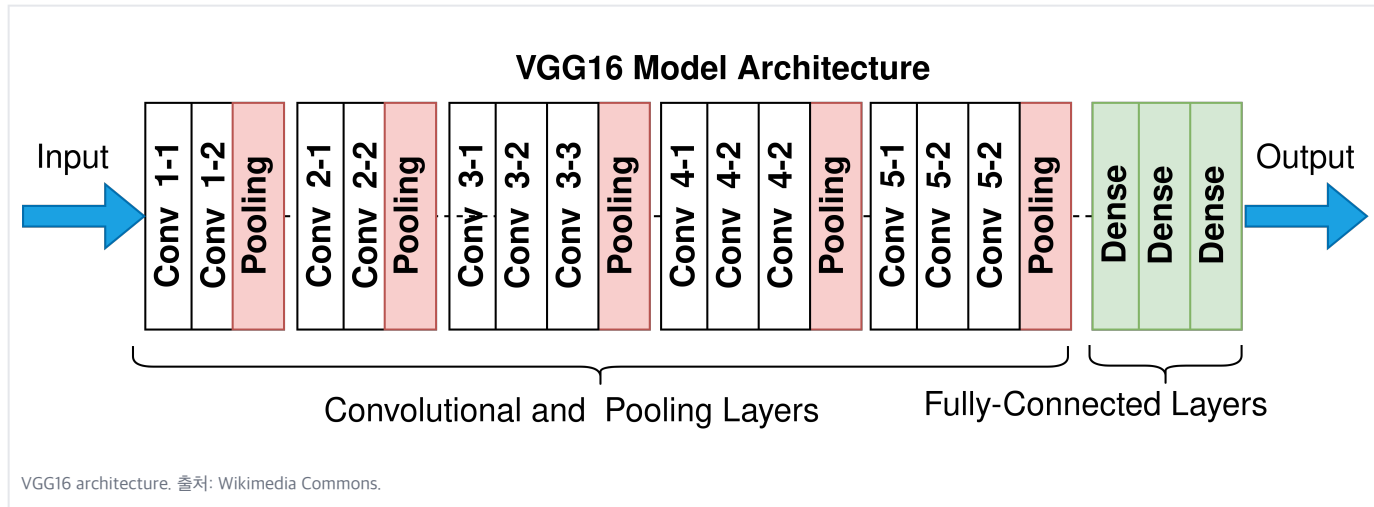
대규모 이미지 분류의 전환점

- AlexNet은 ImageNet 규모의 이미지 분류에서 깊은 CNN의 성능을 크게 부각시킨 모델입니다.
- 논문 구조는 convolution 5개 층과 fully connected 3개 층을 사용했습니다.
- ReLU, dropout, data augmentation, GPU 학습이 함께 중요했습니다.
- 이후 CNN 연구는 더 깊고 안정적인 구조를 향해 발전했습니다.



AlexNet architecture illustration. 출처: Wikimedia Commons.

VGG



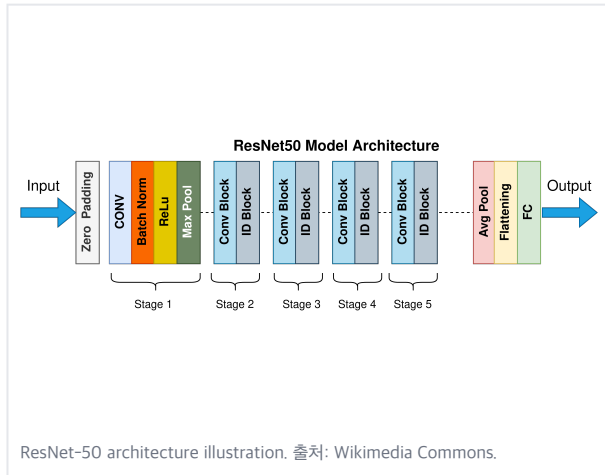
VGG의 핵심 아이디어는 큰 kernel 하나보다 작은 3×3 convolution을 반복해 깊이를 늘리는 것입니다. 구조가 단순해 CNN block을 설명하기 좋지만, parameter와 계산량이 큼니다.

ResNet

깊은 네트워크를 학습시키는 shortcut

- 깊이가 늘면 표현력은 커지지만 gradient 흐름이 어려워질 수 있습니다.
- ResNet은 입력을 block 출력에 더하는 residual connection을 사용합니다.
- 네트워크가 새 정보를 배우기 어렵다면 최소한 identity mapping을 유지할 수 있습니다.
- 매우 깊은 CNN 구조의 학습 안정성을 크게 높였습니다.

$$y = F(x; W) + x$$



Global Average Pooling

feature map을 class score로 연결한다

- CNN 마지막에는 공간 feature map을 vector로 요약해야 합니다.
- Flatten 후 fully connected layer를 쓰면 parameter가 크게 늘 수 있습니다.
- Global Average Pooling은 각 channel의 평균 activation을 하나의 값으로 요약합니다.
- CAM류 해석 기법과도 연결되어 class별 activation 위치를 볼 때 유용합니다.

$$g_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W A_{i,j,c}$$

$$\hat{p} = \text{softmax}(W \text{ GAP}(f_{\theta}(x)) + b)$$

GAP은 공간 정보를 완전히 버리는 대신,
parameter 수와 과적합 위험을 줄이는 효과가 있습니다.

Data Augmentation

이미지에서 일반화를 돕는 핵심 도구

- 같은 label을 유지하는 변환을 학습 중 무작위로 적용합니다.
- 좌우 반전, crop, rotation, color jitter, blur, cutout, mixup 등이 사용됩니다.
- augmentation은 모델이 배경, 위치, 밝기 변화에 덜 민감하도록 돕습니다.
- 단, 의료 영상이나 문서처럼 방향이 의미를 바꾸는 도메인에서는 조심해야 합니다.

$$(x, y) \mapsto (T(x), y), \quad T \sim \mathcal{A}$$

augmentation은 데이터를 “복사”하는 것이 아니라, 학습 분포를 더 넓게 만드는 regularization으로 이해할 수 있습니다.

Transfer Learning

작은 데이터에서 CNN을 쓰는 현실적인 방법

- 대규모 이미지 데이터로 미리 학습된 backbone을 가져옵니다.
- 앞쪽 layer는 edge와 texture처럼 일반적인 feature를 이미 학습했을 가능성이 큼니다.
- 새 task에서는 classifier head만 바꾸거나, 일부 layer를 fine-tuning합니다.
- 데이터가 작을수록 freeze 범위를 크게 잡고, 데이터가 충분할수록 더 많은 layer를 학습합니다.

전략	언제 쓰나	주의
feature extraction	데이터가 매우 작음	domain 차이 크면 한계
head fine-tuning	클래스만 다름	learning rate 작게
full fine-tuning	데이터 충분	과적합과 비용 증가
self-supervised pretrain	label 부족	pretrain 품질 중요

CNN의 과업 확장

분류만 하는 모델이 아니다

- Classification: 이미지 전체의 클래스 예측
- Object Detection: 객체 위치와 클래스를 함께 예측
- Segmentation: 픽셀 단위 클래스 예측
- Image Captioning: 이미지에서 문장을 생성
- CNN backbone은 이런 과업에서 feature extractor로 자주 쓰입니다.



COCO detection examples. 출처: COCO official website/GitHub assets.

CNN 데이터셋 설계

이미지 품질과 label 기준

- 해상도, 압축, blur, 촬영 각도, 조명 조건이 성능을 좌우합니다.
- label granularity를 먼저 정해야 합니다. 예: “고양이”인지 “삼고양이”인지.
- train/test split은 촬영 장소, 사용자, 시간 단위 중복을 피해야 합니다.
- 같은 장면에서 잘라낸 이미지가 train과 test에 동시에 들어가면 leakage가 됩니다.

항목	점검 질문
해상도	작은 객체가 보이는가?
label	모호한 경계 기준이 있는가?
split	같은 인물/장소가 섞였는가?
imbalance	드문 클래스가 충분한가?
privacy	얼굴, 번호판, 민감정보가 있는가?

Evaluation

과업별 지표를 구분한다

- 이미지 분류는 accuracy, top-k accuracy, macro F1을 자주 봅니다.
- detection은 IoU와 mAP가 핵심입니다.
- segmentation은 pixel accuracy보다 IoU, Dice를 더 중요하게 볼 때가 많습니다.
- 클래스 불균형이 있으면 전체 accuracy만으로는 실패를 숨길 수 있습니다.

과업	대표 지표
classification	accuracy, top-k, F1
detection	mAP, IoU, precision/recall
segmentation	mIoU, Dice
retrieval	Recall@K, mAP
production	latency, memory, calibration

CNN 평가는 모델 구조보다 “어떤 오류가 실제로 비용이 큰가”를 먼저 정해야 합니다.

Common Failure Modes

이미지 모델이 자주 실패하는 방식

- 배경 shortcut을 학습합니다.
- 조명, 해상도, 카메라 종류가 바뀌면 성능이 떨어집니다.
- train set에 많은 포즈와 각도만 잘 맞춥니다.
- 작은 객체, occlusion, motion blur에 약합니다.

증상	점검
특정 배경에서만 맞음	saliency/CAM, 배경 crop 테스트
현장 이미지에서 실패	domain shift, 촬영 조건
드문 클래스 실패	class imbalance, sampling
작은 객체 누락	입력 해상도, feature pyramid
과신	calibration, threshold tuning

CNN 설계 기준

결정	질문	기본 방향
입력 크기	작은 객체가 필요한가?	클수록 정보↑, 비용↑
backbone	정확도와 속도 중 무엇이 중요한가?	ResNet, EfficientNet, MobileNet 등
augmentation	label을 보존하는 변환인가?	도메인 규칙 우선
pretrained	유사한 이미지로 학습됐는가?	작을수록 pretrained 유리
metric	실제 비용을 반영하는가?	accuracy만 보지 않기

CNN 실험은 architecture 이름보다 입력 크기, 데이터 품질, augmentation, split 설계가 성능을 더 크게 좌우하는 경우가 많습니다.

핵심 요약

입력

이미지는 위치와 channel을 가진 tensor이며, CNN은 이 구조를 유지합니다.

합성곱

filter가 local patch를 반복해서 보며 feature map을 만듭니다.

효율

parameter sharing 덕분에 FC보다 훨씬 적은 parameter로 이미지 패턴을 학습합니다.

깊이

앞쪽 layer는 edge와 texture, 뒤쪽 layer는 part와 object-level pattern을 학습합니다.

일반화

augmentation, transfer learning, 좋은 split이 CNN 성능의 핵심입니다.

평가

classification, detection, segmentation마다 지표와 오류 비용이 다릅니다.

참고자료

- LeCun et al., “Gradient-Based Learning Applied to Document Recognition,” 1998: <http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf>
- Krizhevsky, Sutskever, Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” 2012: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>
- Simonyan, Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” 2014: <https://arxiv.org/abs/1409.1556>
- He et al., “Deep Residual Learning for Image Recognition,” 2015: <https://arxiv.org/abs/1512.03385>

이미지 출처

- Wikimedia Commons, Convolutional Network (vector) .svg, Will Whitney, CC BY-SA 4.0: [https://commons.wikimedia.org/wiki/File:Convolutional_Network_\(vector\).svg](https://commons.wikimedia.org/wiki/File:Convolutional_Network_(vector).svg)
- Wikimedia Commons, convolution/maxpooling examples, Daniel Voigt Godoy, CC BY 4.0: https://commons.wikimedia.org/wiki/File:Convolutional_neural_network,_convolution_worked_example.png
- Wikimedia Commons, Typical_cnn.png: https://commons.wikimedia.org/wiki/File:Typical_cnn.png
- Wikimedia Commons, LeNet-5_architecture.svg: https://commons.wikimedia.org/wiki/File:LeNet-5_architecture.svg
- Wikimedia Commons, Alexnet.svg, VGG16.png, ResNet50.png: <https://commons.wikimedia.org/>
- COCO official website/GitHub assets: <https://cocodataset.org/>
- Wikimedia Commons, MnistExamples.png, Josef Steppan, CC BY-SA 4.0: <https://commons.wikimedia.org/wiki/File:MnistExamples.png>